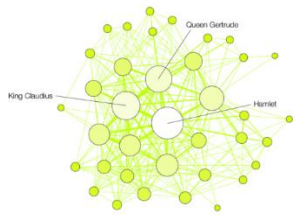
CS 225

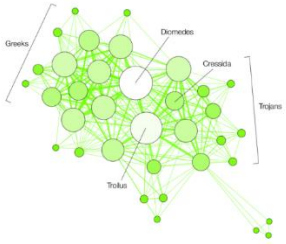
#32: Introduction to Graphs

May 6-8, 2026 V. Kindratenko

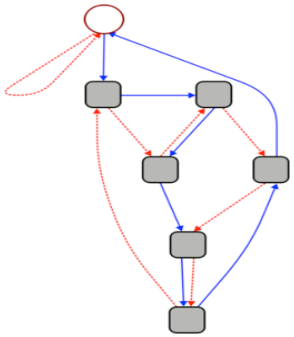
An Introduction to Graphs

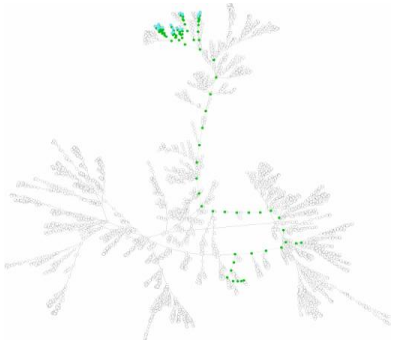


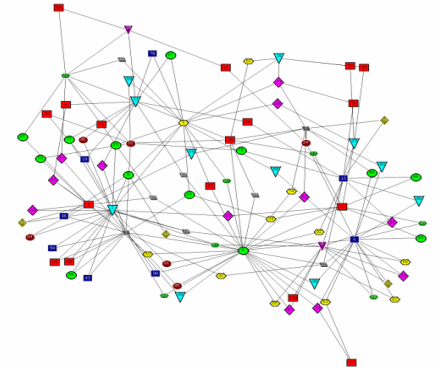
HAMLET

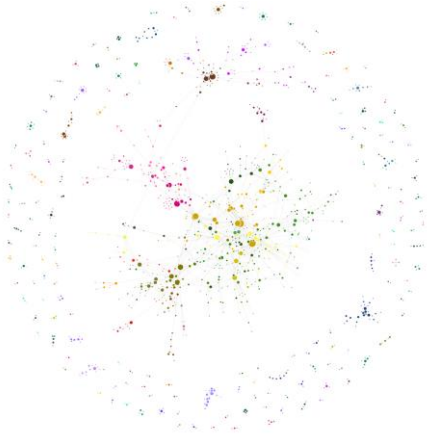


TROILUS AND CRESSIDA









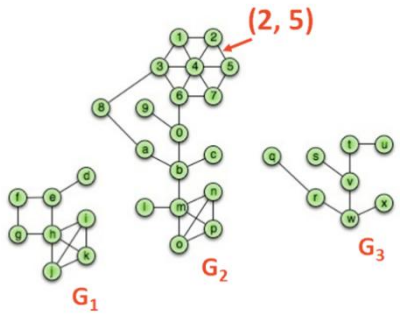
Motivation:

Graphs are awesome data structures that allow us to represent an enormous range of problems. To study these problems, we need:

1. A common vocabulary to talk about graphs
2. Implementation(s) of a graph
3. Traversals on graphs
4. Algorithms on graphs

Graph Vocabulary

Consider a graph **G** with vertices **V** and edges **E**, **G=(V,E)**.



Incident Edges:
I(v) = { (x, v) in E }

Degree(v): **|I|**

Adjacent Vertices:
A(v) = { x : (x, v) in E }

Path(G_2): Sequence of vertices connected by edges

Cycle(G_1): Path with a common begin and end vertex.

Simple Graph(G): A graph with no self loops or multi-edges.

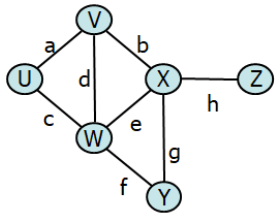
Subgraph(G): **G' = (V', E')**:
V' ∈ V, E' ∈ E, and (u, v) ∈ E → u ∈ V', v ∈ V'

Graphs that we will study this semester include:
- Complete subgraph(G)
- Connected subgraph(G)
- Connected component(G)
- Acyclic subgraph(G)
- Spanning tree(G)

Size and Running Times

Running times are often reported by **n**, the number of vertices, but often depend on **m**, the number of edges.

For arbitrary graphs, the minimum number of edges given a graph that is:



Not Connected:

Minimally Connected:*

The maximum number of edges given a graph that is:

Simple:

Not Simple:

The relationship between the degree of the graph and the edges:

Proving the Size of a Minimally Connected Graph

Theorem: Every minimally connected graph **G=(V, E)** has **|V|-1** edges.

Proof of Theorem
Consider an arbitrary, minimally connected graph **G=(V, E)**.

Lemma 1: Every connected subgraph of **G** is minimally connected. *(Easy proof by contradiction left for you.)*

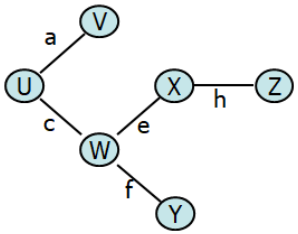
Inductive Hypothesis: For any **j < |V|**, any minimally connected graph of **j** vertices has **j-1** edges.

Suppose |V| = 1:
Definition: A minimally connected graph of 1 vertex has 0 edges.

Theorem: |V|-1 edges ➔ 1-1 = 0.

Suppose |V| > 1:
Choose any vertex **u** and let **d** denote the degree of **u**.

Remove the incident edges of **u**, partitioning the graph into ____ components: $\mathbf{C_0 = (V_0, E_0)}$, …, $\mathbf{C_d = (V_d, E_d)}$.



By Lemma 1, every component $\mathbf{C_k}$ is a minimally connected subgraph of **G**.

By our _____:

Finally, we count edges:

Graph ADT

Data	Functions

Vertices
 	
Edges
 	
Some data structure maintaining the structure between vertices and edges.
	`insertVertex(K key);`
	`insertEdge(Vertex v1, Vertex v2, K key);`
	
	`removeVertex(Vertex v);`
	`removeEdge(Vertex v1, Vertex v2);`
	
	`incidentEdges(Vertex v);`
	`areAdjacent(Vertex v1, Vertex v2);`
	
	`origin(Edge e);`
	`destination(Edge e);`

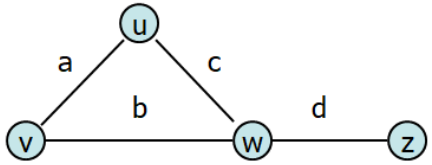
CS 225

#33: Graph Implementations

May 6-8, 2026 V. Kindratenko

Graph Implementation #1: Edge List

Vert.	Edges
u v w z	a b c d



Operations:

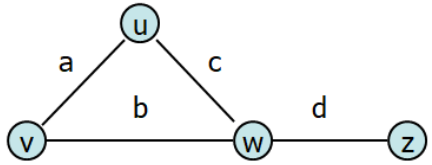
insertVertex(K key):

removeVertex(Vertex v):

areAdjacent(Vertex v1, Vertex v2):

incidentEdges(Vertex v):

Graph Implementation #2: Adjacency Matrix



Vert.	Edges	Adj. Matrix
u v w z	a b c d	

	u	v	w	z
u				
v				
w				
z				

Operations on an Adjacency Matrix:

insertVertex(K key):

removeVertex(Vertex v):

areAdjacent(Vertex v1, Vertex v2):

incidentEdges(Vertex v):

Graph Implementation #3: Adjacency List

Vertex List	Edges
u v w z	a b c d

Operations on an Adjacency List:

insertVertex(K key):

removeVertex(Vertex v):

areAdjacent(Vertex v1, Vertex v2):

incidentEdges(Vertex v):

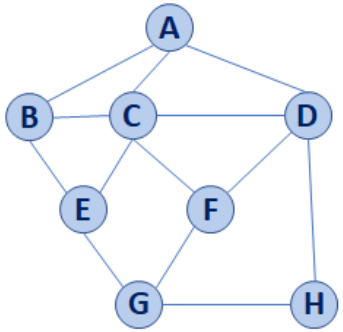
Running Times of Classical Graph Implementations

	Edge List	Adj. Matrix	Adj. List
Space	**n+m**	**n^2**	**n+m**
insertVertex	**1**	**n**	**1**
removeVertex	**m**	**n**	**deg(v)**
insertEdge	**1**	**1**	**1**
removeEdge	**1**	**1**	**1**
incidentEdges	**m**	**n**	**deg(v)**
areAdjacent	**m**	**1**	**min(deg(v), deg(w))**

How do the algorithms compare?

...is one clearly better?

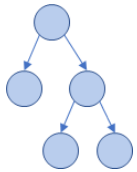
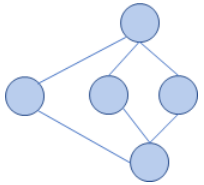
BST Graph Traversal



Graph Traversal

Objective: Visit every vertex and every edge in the graph.
Purpose: Search for interesting sub-structures in the graph.

We've seen traversal before – this is different:

BST	Graph
	

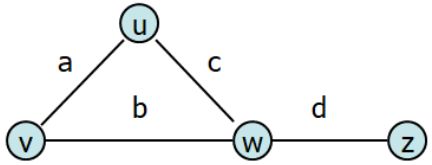
CS 225

#34: Graph Traversal – BFS

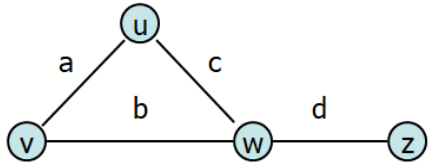
May 6-8, 2025 V. Kindratenko

Graph Implementation Review #1: Edge List

Vert.	Edges
u v w z	a b c d



Graph Implementation Review #2: Adjacency Matrix



Vert.	Edges	Adj. Matrix
u v w z	a b c d	

	u	v	w	z
u				
v				
w				
z				

Graph Implementation Review #3: Adjacency List

Vertex List	Edges
u v w z	a b c d

Running Times of Classical Graph Implementations

	Edge List	Adj. Matrix	Adj. List
Space	n+m	n^2	n+m
insertVertex	1	n	1
removeVertex	m	n	deg(v)
insertEdge	1	1	1
removeEdge	1	1	1
incidentEdges	m	n	deg(v)
areAdjacent	m	1	min(deg(v), deg(w))

Implementations and Use Cases

Ex. 1:

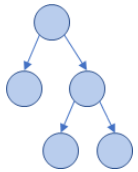
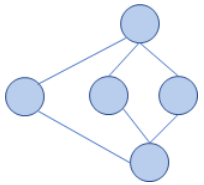
Ex. 2:

Ex. 3:

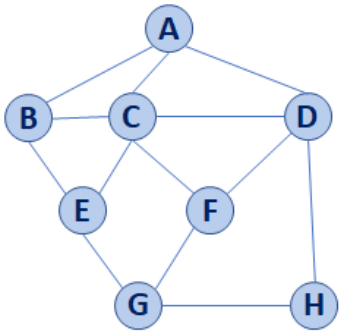
Graph Traversal

Objective: Visit every vertex and every edge in the graph.
Purpose: Search for interesting sub-structures in the graph.

We've seen traversal before – this is different:

BST	Graph
	

BFS Graph Traversal



A	
B	
C	
D	
E	
F	
G	
H	

Pseudocode for BFS

```
1   BFS(G):
2     Input: Graph, G
3     Output: A labeling of the edges on
4         G as discovery and cross edges
5
6     foreach (Vertex v : G.vertices()):
7       setLabel(v, UNEXPLORED)
8     foreach (Edge e : G.edges()):
9       setLabel(e, UNEXPLORED)
10    foreach (Vertex v : G.vertices()):
11      if getLabel(v) == UNEXPLORED:
12        BFS(G, v)
13
14  BFS(G, v):
15    Queue q
16    setLabel(v, VISITED)
17    q.enqueue(v)
18
19    while !q.empty():
20      v = q.dequeue()
21      foreach (Vertex w : G.adjacent(v)):
22        if getLabel(w) == UNEXPLORED:
23           setLabel(v, w, DISCOVERY)
24           setLabel(w, VISITED)
25           q.enqueue(w)
26        elseif getLabel(v, w) == UNEXPLORED:
27           setLabel(v, w, CROSS)
```

CS 225 – Things To Be Doing:

1. mp6 due Monday, May 11
2. mp7 Extra credit deadline is Monday, May 18
3. Quiz 6 on May 7
4. lab_dict due this Sunday, May 10